AWS

AWS Cloud Computing

Complete Study Guide

Serverless | Storage | Compute | Networking | Infrastructure

10 Questions · Architecture Diagrams · Comparison Tables

AWS Lambda · S3 · EBS · ECS · Fargate · Elastic Beanstalk · EC2

Amazon Web Services · Cloud Architecture Reference · 2024

TABLE OF CONTENTS

No.	Topic	Coverage
Q1	Serverless Computing — Lambda vs Fargate	Scalability, Dev Productivity, Cost
Q2	Amazon SQS Architecture & Durability	Architecture Diagram + Durability Mechanisms
Q3	AWS Elastic Beanstalk in Modern DevOps	Role, Benefits, Workflow
Q4	S3 vs EBS — Features & Use Cases	Comparison Table
Q5	Storage Type Selection by Workload	Decision Framework
Q6	ECS + Fargate — How They Work Together	Container Architecture
Q7	Fargate vs EC2 — When to Choose Fargate	Decision Criteria
Q8	Configure & Manage Files in S3 Bucket	Step-by-Step Guide
Q9	Cloud Elasticity for E-Commerce Traffic Spikes	Festival Sale Scenario
Q10	EC2 Pricing for Unpredictable Startup Traffic	Instance & Pricing Model Evaluation
Q11	EBS vs S3 Differences	Feature Comparison
Q12	Cloud Computing vs Traditional On-Premise	Comparative Analysis
Q13	AWS Global Infrastructure — Multi-Layer	Latency, DR, Availability
Q14	Launching EC2 Instance with PuTTY	Step-by-Step SSH Guide
Q15	XYZ Online Platform — AWS Solution Design	Complete Architecture for Exam Platform

Q1. Serverless Computing — Evaluate Lambda and Fargate w.r.t Scalability, Dev Productivity & Cost

What is Serverless Computing?

Serverless computing allows developers to build and run applications without managing servers. The cloud provider automatically provisions, scales, and manages infrastructure. AWS offers two primary serverless compute options: **AWS Lambda** (function-based) and **AWS Fargate** (container-based).



↑ *Typical Serverless Request Flow: Client → API Gateway → Lambda/Fargate → Backend Services*

Comparison: AWS Lambda vs AWS Fargate

Dimension	AWS Lambda	AWS Fargate
Model	Event-driven FaaS (Function as a Service)	Container-based CaaS (Container as a Service)
Scalability	Auto-scales to 1000s of concurrent executions instantly. Scales per request.	Scales container tasks; slightly slower cold start but handles long-running workloads.
Dev Productivity	High — write only business logic. No Dockerfiles. Rapid iteration.	Medium — requires container image build & push. More control over runtime.
Max Execution Time	15 minutes per invocation	Unlimited (long-running containers)
Cost Model	Pay per 100ms execution + requests. Free tier: 1M req/month.	Pay per vCPU/memory per second. No free tier but cost-effective for sustained load.
Best For	Short tasks, APIs, event processing, scheduled jobs	Microservices, ML inference, long-running tasks, legacy containerized apps
Cold Start	Yes (ms to seconds)	Yes (seconds, container startup)
State Management	Stateless — use S3/DynamoDB for state	Stateless containers — use EFS/RDS for persistence

Key Takeaways

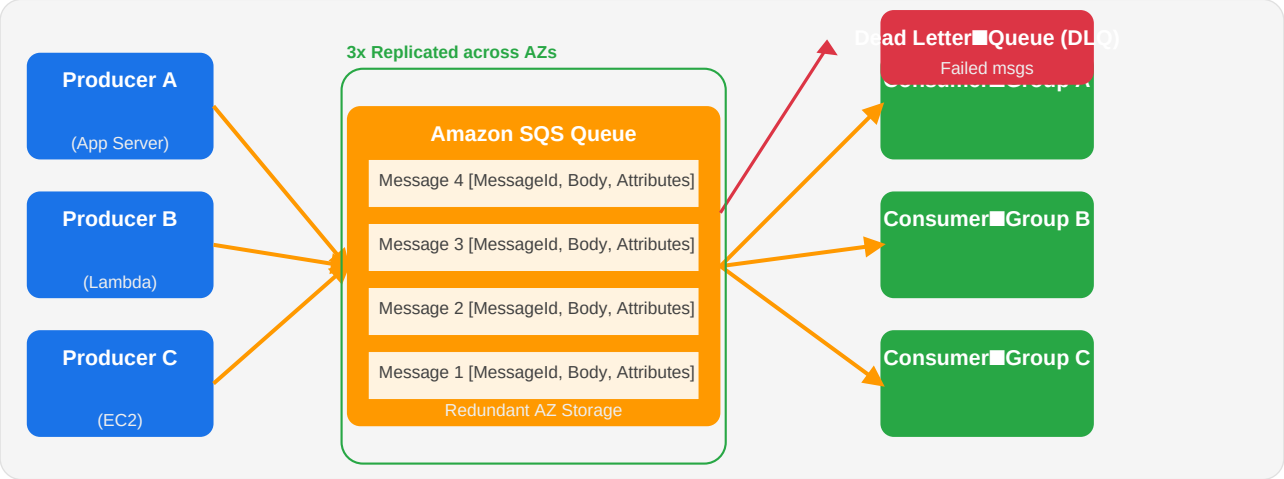
- Lambda excels for event-driven microservices, APIs, and bursty workloads with zero idle cost.
- Fargate suits containerized workloads needing custom runtimes, longer execution, or fine-grained resource control.
- Both eliminate server management — Lambda abstracts more; Fargate gives more control.
- For mixed workloads: use Lambda for lightweight async tasks + Fargate for heavyweight processing.

Q2. Architecture of Amazon SQS — Highlight How Durability is Achieved

Amazon SQS Overview

Amazon Simple Queue Service (SQS) is a fully managed message queuing service that enables decoupling of microservices, distributed systems, and serverless applications. Messages are stored reliably until processed.

SQS Architecture Diagram



How Durability is Achieved in SQS

- **Multi-AZ Redundancy:** Every message is stored redundantly across multiple Availability Zones within the region.
- **Server-Side Encryption (SSE):** Messages encrypted at rest using AWS KMS keys.
- **Message Retention:** Messages retained for up to 14 days, protecting against consumer failures.
- **Visibility Timeout:** Message hidden from other consumers while being processed; re-appears if not deleted — prevents loss.
- **Dead Letter Queue (DLQ):** Failed messages after N retries move to DLQ for investigation, ensuring no silent drops.
- **At-Least-Once Delivery:** Standard queues guarantee delivery; FIFO queues add exactly-once processing.
- **Long Polling:** Reduces empty receives and ensures messages are picked up reliably.
- **Message Attributes & Deduplication (FIFO):** MessageDeduplicationId prevents duplicate processing.

SQS Type	Ordering	Deduplication	Throughput	Use Case
Standard Queue	Best-effort	No	Unlimited	High-throughput, order not critical
FIFO Queue	Strict FIFO	Yes (5 min window)	3,000 msg/s with batching	Financial, ordering, inventory systems

Q3. Analyse the Role of Elastic Beanstalk in Modern DevOps Practices

What is AWS Elastic Beanstalk?

Elastic Beanstalk (EB) is a PaaS (Platform as a Service) that automates deployment, capacity provisioning, load balancing, auto-scaling, and monitoring of applications. Developers upload code; EB handles the rest.



↑ *Beanstalk DevOps Flow: Code push → Auto-provision → Deploy → Monitor*

Role in Modern DevOps

DevOps Practice	Beanstalk Contribution
CI/CD Integration	Integrates with CodePipeline, CodeBuild, GitHub Actions for automated deployments
Infrastructure as Code	Environment configs via .ebextensions YAML files stored in source control
Blue/Green Deployments	Swap environment URLs for zero-downtime releases
Rolling Updates	Deploy new version to batches of instances progressively
Auto-Scaling	Triggers scale-out/in based on CPU, requests, custom CloudWatch metrics
Monitoring	Built-in health dashboard; CloudWatch integration; SNS alerts
Multi-Environment	Separate dev, staging, prod environments from single codebase
Managed Platform Updates	AWS patches runtime (Java, Python, Node.js, Docker) automatically

Supported Platforms & Deployment Strategies

Platform	Strategy	Description
Java, .NET, Node.js PHP, Ruby, Python, Go	All-at-once	Fast; downtime risk — dev/test only
Docker, Multi-container	Rolling	Batches updated; no downtime; slower
Custom AMI	Rolling with batch	Extra capacity added; expensive
Preconfigured	Blue/Green	New env spun up; zero downtime; safest for prod

Key Benefits & Limitations

- ■ Zero infra management — focus on code
- ■ Fast onboarding — deploy in minutes
- ■ Free (pay only for underlying resources)
- ■ Full control still available via SSH/console
- ■ Less flexible than raw CloudFormation/CDK
- ■ Not ideal for highly custom architectures
- ■ Platform version lag vs direct AMI control

Q4. Differentiate Between S3 and EBS in Terms of Features and Use Cases



Feature	Amazon S3	Amazon EBS
Type	Object Storage	Block Storage
Access	HTTP/S REST API; accessible from anywhere	Mounted as disk to single EC2 instance (Multi-Attach GP for EBS io2)
Durability	99.999999999% (11 nines) — 3+ AZ copies	99.8–99.9%; within single AZ (snapshots for backup)
Scalability	Virtually unlimited (exabyte scale)	Limited by volume size (up to 64 TB per volume)
Latency	ms to seconds (network-based)	Sub-ms (attached block device)
Data Model	Objects (key-value with metadata)	Raw blocks / filesystem
Pricing	~$0.023/GB/month (Standard)	~$0.08–0.125/GB/month (gp3)
Backup / DR	Cross-region replication, versioning	Snapshots to S3
Encryption	SSE-S3, SSE-KMS, SSE-C, client-side	AES-256, AWS KMS
Use Cases	Static files, backups, data lake, ML datasets, media hosting	OS disk, databases, transactional apps needing low-latency I/O

Rule of Thumb: Use EBS for 'drive-like' storage attached to EC2. Use S3 for 'bucket-like' storage accessible from anywhere, at scale.

Q5. Analyse How Workload Characteristics Determine Storage Type Selection

AWS Storage Services Overview

Amazon S3 — Object Storage

Static files, backups, data lakes, 11-nines durability

Amazon EBS — Block Storage

EC2-attached disks, DBs, transactional I/O, <1ms latency

Amazon EFS — File Storage

Shared NFS across multiple EC2s, scalable, POSIX-compliant

Amazon Glacier — Archival Storage

Long-term archive, retrieval hours, lowest cost

Amazon FSx — Managed File System

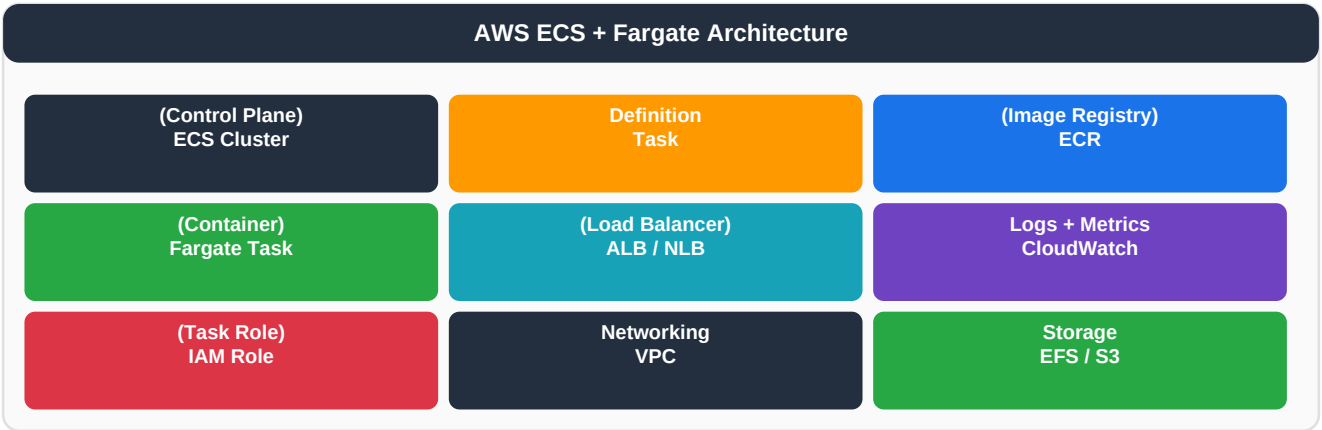
Windows FSx (SMB/NTFS), Lustre (HPC/ML workloads)

Workload-to-Storage Decision Framework

Workload Characteristic	Best Storage	Reason
High IOPS, low latency (database)	EBS io2/gp3	Block storage with <1ms, up to 64K IOPS
Shared file access (multiple EC2s)	EFS	NFS protocol, auto-scales, multi-AZ
Static website / media files	S3	HTTP access, CDN integration, unlimited scale
Infrequent backups / archives	S3 Glacier	Lowest cost, retrieval in hours/days
HPC / ML training datasets	FSx for Lustre	Parallel file system, millions of IOPS
Windows workloads / SMB shares	FSx for Windows	Native SMB/NTFS, AD integration
Object metadata + analytics	S3 + Athena	SQL queries on S3 data
Boot volume for EC2	EBS gp3	Required block device for OS
Containerized ephemeral storage	Instance Store	Temporary, highest IOPS, deleted on stop

Q6. Analyse How ECS and Fargate Work Together

ECS Architecture with Fargate



How They Work Together

Component	Role	Detail
ECS Cluster	Logical grouping	Manages tasks & services; control plane for orchestration
Task Definition	Blueprint	Defines CPU/memory, Docker image URI, env vars, IAM role, port mappings
Fargate Launch Type	Serverless infra	AWS provisions ENI, CPU/RAM — no EC2 instance management
ECS Service	Desired state	Maintains N running tasks; integrates with ALB for traffic distribution
ECR	Image registry	Stores Docker images; integrated auth with ECS; vulnerability scanning
ALB	Traffic routing	Routes HTTP/HTTPS to healthy task IPs using target groups
CloudWatch	Observability	Container Insights: CPU, memory, task failures, custom metrics
AWS VPC	Networking	Each Fargate task gets private IP in VPC subnet; security groups apply

Deployment Flow (Step by Step)

1. Build Docker image → push to ECR
2. Create Task Definition (image, CPU, memory, env vars)
3. Create ECS Cluster (Fargate launch type)
4. Create ECS Service → attach ALB Target Group
5. ECS launches Fargate tasks in VPC subnets
6. ALB routes traffic → health checks → CloudWatch monitors

Q7. When Should an Organisation Select Fargate Instead of EC2?

Criteria	Choose Fargate	Choose EC2 Launch Type
Infrastructure Management	Want zero server management	OK managing EC2 instances, AMIs, patching
Workload Pattern	Variable/spiky — pay per task	Sustained/predictable — Reserved Instances save cost
Cost Optimization	Bursty; no idle server cost	High steady load — EC2 RI cheaper
Custom Kernel / GPU	Not needed	Need GPU (ML), custom OS, specific kernel settings
Compliance / Security	Standard AWS isolation	Need dedicated hosts (HIPAA, PCI strict isolation)
Startup Speed	Acceptable (30-60s)	Need faster task placement (pre-warmed EC2)
Density / Packing	Not a concern	Need bin-packing many small containers on fewer hosts
EBS Volumes	Not supported directly (EFS only)	EBS volumes attachable to EC2-backed tasks

Decision Flowchart



Summary Recommendation

- Choose **Fargate** for: startups, variable traffic, microservices, rapid deployment, zero-ops philosophy.
- Choose **EC2 launch type** for: GPU workloads, steady-state high-throughput, compliance needing dedicated hosts, tight cost control.
- Hybrid approach: Use Fargate for web tier (variable) + EC2 for ML/processing tier (GPU/sustained).

Q8. Step-by-Step: Configure, Upload & Manage Files in an S3 Bucket (Including Settings & Permissions)

Phase 1 — Create & Configure S3 Bucket

Step	Action	Details
1	Navigate to S3 Console	AWS Console → Services → S3 → Create bucket
2	Bucket Name & Region	Globally unique name (e.g., myapp-assets-2024); select nearest region
3	Block Public Access	Leave ALL blocked for private buckets; unblock selectively for static websites
4	Versioning	Enable for critical data — preserves all object versions; recovery from accidental deletes
5	Server-Side Encryption	Select SSE-S3 (default) or SSE-KMS for key management compliance
6	Object Lock	Enable for WORM compliance (healthcare, financial records)
7	Tags	Add key-value tags: Project=MyApp, Env=Prod for cost allocation
8	Create Bucket	Click 'Create bucket' — takes seconds

Phase 2 — Set Permissions & Policies

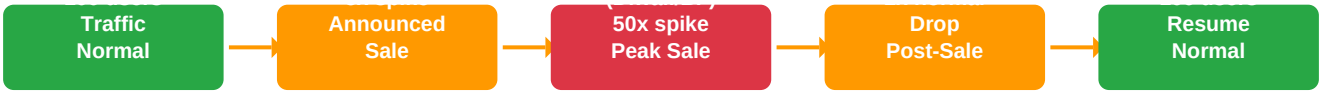
Permission Type	How to Configure	Example
Bucket Policy	S3 → Bucket → Permissions → Bucket Policy → JSON editor	{"Effect":"Allow","Principal":"*","Action":"s3:GetObject","Resource":"arn:aws:s3:::bucket/*"}
IAM Policy	IAM → Roles/Users → Attach Policy → AmazonS3FullAccess or custom	Grant EC2 role s3:PutObject on specific bucket ARN
ACL	S3 → Permissions → ACL → Grant bucket-owner / authenticated users	Legacy; prefer bucket policies
CORS Policy	S3 → Permissions → CORS → JSON config	Allow web app at domain to fetch S3 objects
Presigned URL	Generate via SDK/CLI for temporary access (15min–7days)	Share private files without making bucket public

Phase 3 — Upload & Manage Files

- **Console Upload:** S3 → Bucket → Upload → Drag files → Set storage class → Upload
- **AWS CLI Upload:** aws s3 cp file.txt s3://bucket-name/ --storage-class STANDARD_IA
- **Sync folder:** aws s3 sync ./local-folder s3://bucket-name/prefix/ --delete
- **Multipart Upload:** For files >100MB — automatic via CLI/SDK; supports parallel upload
- **Storage Classes:** Standard (hot), IA (infrequent), Glacier (archive) — assign per object or lifecycle
- **Lifecycle Policy:** Auto-transition objects: Standard → IA after 30d → Glacier after 90d → Delete after 365d
- **Cross-Region Replication:** Enable for DR — automatically replicates to another region bucket
- **S3 Event Notifications:** Trigger Lambda/SQS/SNS on PutObject, DeleteObject, etc.

Q9. Cloud Elasticity for E-Commerce: Handling Festival Sale Traffic Spikes

The Problem — Festival Sale Traffic Pattern



> *Without elasticity: servers crash at peak. With elasticity: capacity matches demand automatically.*

AWS Elastic Architecture for E-Commerce

CloudFront CDN — Cache static assets; absorb up to 90% of traffic at edge

HTML/CSS/JS/Images cached globally in 400+ PoPs

Application Load Balancer — Distribute HTTP requests across Auto Scaling Group

Health checks; path-based routing; SSL termination

EC2 Auto Scaling Group — Min 2, Desired 4, Max 100 instances (scale on CPU >60%)

Launch template; warm pools for pre-warmed capacity

ElastiCache (Redis) — Cache product listings, sessions, cart data

Reduces DB load by 80%+; sub-ms responses

Amazon RDS Multi-AZ + Read Replicas — Primary writes; read replicas for catalog queries

Auto failover; up to 5 read replicas for MySQL/PostgreSQL

Amazon S3 + SQS — Static assets; async order processing queue

Decouple checkout from fulfillment; no order loss

Elasticity Mechanisms Used

Service	Elasticity Mechanism	Scale Trigger
EC2 Auto Scaling	Scale Out/In	CPU > 60% → add instances; CPU < 20% → remove
CloudFront	Global CDN	Auto-distributes globally, no configuration needed
RDS Read Replicas	Horizontal reads	Add replicas manually before sale event
ElastiCache	Cluster mode	Shard-based auto-scaling for Redis
Lambda (checkout)	Per-request scaling	Zero idle cost; instant scale to 10K concurrent
SQS	Infinite queue	Absorbs order bursts; consumers scale independently

Q10. Evaluate Suitable EC2 Instance Types & Pricing Models for a Startup with Unpredictable Traffic

Recommended EC2 Instance Families

Instance Family	vCPU	RAM	Best For	Example
t3/t4g (Burstable)	2–8	1–32 GB	Web servers, dev, bursty workloads	t3.medium (2vCPU, 4GB)
m6i/m7i (General)	2–128	8–512 GB	Balanced web/app servers	m6i.large (2vCPU, 8GB)
c6i (Compute-opt)	2–96	4–192 GB	CPU-heavy: encoding, inference	c6i.xlarge (4vCPU, 8GB)
r6i (Memory-opt)	2–96	16–768 GB	In-memory DBs, caching	r6i.large (2vCPU, 16GB)

EC2 Pricing Models — Comparison

Pricing Model	Savings vs On-Demand	Commitment	Best For Startup?
On-Demand	Baseline (0%)	None — pay by hour/second	■ Yes — flexibility, no risk
Spot Instances	Up to 90% off	Can be interrupted 2-min notice	■ For fault-tolerant batch, CI/CD
Reserved Instances (1yr)	~30–40% off	1-year commitment	■ Only if baseline load is predictable
Reserved Instances (3yr)	~60% off	3-year commitment	■ Too rigid for early startup
Savings Plans	Up to 66% off	1–3yr compute commitment	■ If >6 months usage data available
Dedicated Hosts	Premium pricing	1-year+	■ Only for strict compliance needs

Recommended Strategy for Startup

- **Phase 1 (0–6 months):** 100% On-Demand t3.medium/large — no commitment, iterate fast.
- **Phase 2 (6–12 months):** Add Spot Instances for background jobs (image processing, analytics).
- **Phase 3 (12+ months):** Purchase 1-year Savings Plan for proven baseline; Spot for overflow.
- **Auto Scaling:** Always pair EC2 with ASG — scale in/out automatically, pay only for what you use.
- **Cost Monitoring:** Enable AWS Cost Explorer + Budgets alerts at 80% of monthly limit.

Q11. Difference Between EBS and S3

Quick Reference Comparison

Aspect	EBS (Elastic Block Store)	S3 (Simple Storage Service)
Storage Type	Block (raw disk blocks)	Object (files with metadata)
Access Protocol	Attached to EC2 as /dev/xvda device	REST API, AWS CLI, SDK
Attachment	Single EC2 (io2 Multi-Attach: multi-EC2)	Any service, any region, internet
Performance	<1ms latency, up to 64,000 IOPS	ms–seconds latency; throughput-focused
Max Size	64 TB per volume	Unlimited (object max 5 TB)
Durability	99.8–99.9% (single AZ)	99.999999999% (multi-AZ)
Cost (approx)	$0.08–0.125/GB/month	$0.023/GB/month (Standard)
Encryption	AES-256, KMS	SSE-S3, SSE-KMS, SSE-C
Backup	EBS Snapshots → S3	S3 Versioning, Cross-Region Replication
Typical Use Case	OS boot vol, MySQL, PostgreSQL	Images, videos, backups, data lake, static site

Q12. Cloud Computing vs Traditional On-Premise — Comparative Analysis

Dimension	Traditional On-Premise	Cloud Computing (AWS)
Capital Cost	High upfront (servers, racks, networking)	Zero CapEx — pay-as-you-go OpEx
Scaling	Manual; purchase hardware months ahead	Auto-scaling in minutes
Maintenance	Internal IT team patches hardware/OS	AWS manages hardware; shared responsibility
Availability	Depends on single datacenter redundancy	Multi-AZ, multi-region, 99.99%+ SLA
Disaster Recovery	Expensive DR site; manual failover	Automated cross-region DR; Route 53 failover
Security	Physical security control; own compliance	AWS compliance certs (SOC2, ISO, PCI, HIPAA)
Deployment Speed	Weeks to months (hardware procurement)	Minutes (AMI launch, Terraform provision)
Innovation	Slow — constrained by owned hardware	Access to 200+ services (AI/ML, IoT, analytics)
Geographic Reach	Limited to owned datacenters	33 regions, 105 AZs, 400+ edge locations
Cost Predictability	Fixed (after purchase)	Variable — monitor with Cost Explorer

Verdict: Cloud wins for agility, cost-efficiency, and innovation. On-premise may suit regulated industries needing full data sovereignty or ultra-low-latency edge workloads.

Q13. Describe Multi-Layer AWS Global Infrastructure — Latency, Disaster Recovery & Availability

AWS Infrastructure Hierarchy

Layer 1: Edge Locations (400+ globally) — CloudFront CDN, Route 53, Lambda@Edge

Serve cached content <10ms from users; DNS resolution; edge compute

Layer 2: Local Zones (35+) — Single-digit ms latency for metro areas

Run compute/storage in metro cities; gaming, media, real-time apps

Layer 3: Availability Zones (105+ AZs) — Physically separate DCs in a region

Isolated failure domains; high-speed private fiber interconnect

Layer 4: AWS Regions (33 globally) — Geographic clusters of 2–6 AZs

Data sovereignty boundary; region-specific compliance

Layer 5: AWS Global Backbone — Dedicated fiber network between all regions

Private AWS network (not public internet); low-latency inter-region traffic

How Each Layer Contributes

Layer	Latency Reduction	Disaster Recovery	Service Availability
Edge Locations	Cache at edge <10ms from user	N/A (stateless cache)	CloudFront SLA 99.9%
Local Zones	<10ms for metro users	Not for DR — single zone	Extends region capacity
Availability Zones	Low latency within region (<1ms inter-AZ)	Multi-AZ = survive AZ failure	99.99% with Multi-AZ
Regions	Separate for different user bases	Cross-region DR: RPO minutes, RTO minutes	99.99% with multi-region
Global Backbone	Faster than public internet	Region-to-region private routing	AWS backbone SLA

Disaster Recovery Strategies (Cost vs RTO/RPO)

Strategy	RTO	RPO	Cost	How
Backup & Restore	Hours	Hours	$	S3 backups + restore to new region
Pilot Light	10-30 min	Minutes	$$	Minimal replica runs idle; scale on DR
Warm Standby	Minutes	Seconds	$$$	Scaled-down copy runs; promote on DR
Multi-Site Active-Active	<1 min	Near-zero	$$$$	Full capacity in 2+ regions; Route 53 routing

Q14. Steps for Launching an EC2 Instance and Connecting via PuTTY

Part A — Launch EC2 Instance

Step	Action	Detail
1	Open EC2 Console	AWS Console → EC2 → Instances → Launch Instances
2	Choose AMI	Select Amazon Linux 2023 (free-tier eligible) or Ubuntu 22.04
3	Select Instance Type	t2.micro (free tier) → Next: Configure Instance Details
4	Configure Network	Select VPC and subnet; enable Auto-assign Public IP
5	Configure Storage	Default 8GB gp3 root volume (increase if needed)
6	Add Tags	Key: Name, Value: MyEC2Instance
7	Security Group	Create new SG: allow SSH (port 22) from your IP only
8	Create Key Pair	Select 'Create new key pair' → RSA → .ppk format (for PuTTY) → Download and SAVE securely
9	Launch	Click Launch Instance → note the Instance ID

Part B — Connect via PuTTY (Windows)

Step	Action	Detail
1	Download PuTTY	https://putty.org → Download putty.exe and puttygen.exe
2	Convert Key (if .pem)	Open PuTTYgen → Load .pem file → Save private key → .ppk
3	Open PuTTY	Launch putty.exe
4	Enter Host	Connection → Host Name: ec2-user@ (or ubuntu@ for Ubuntu)
5	Load PPK Key	Connection → SSH → Auth → Credentials → Browse .ppk file
6	Save Session	Session tab → Saved Sessions → type name → Save
7	Connect	Click Open → Accept host key → You are now in the EC2 terminal
8	Verify Connection	Run: whoami → should show ec2-user or ubuntu

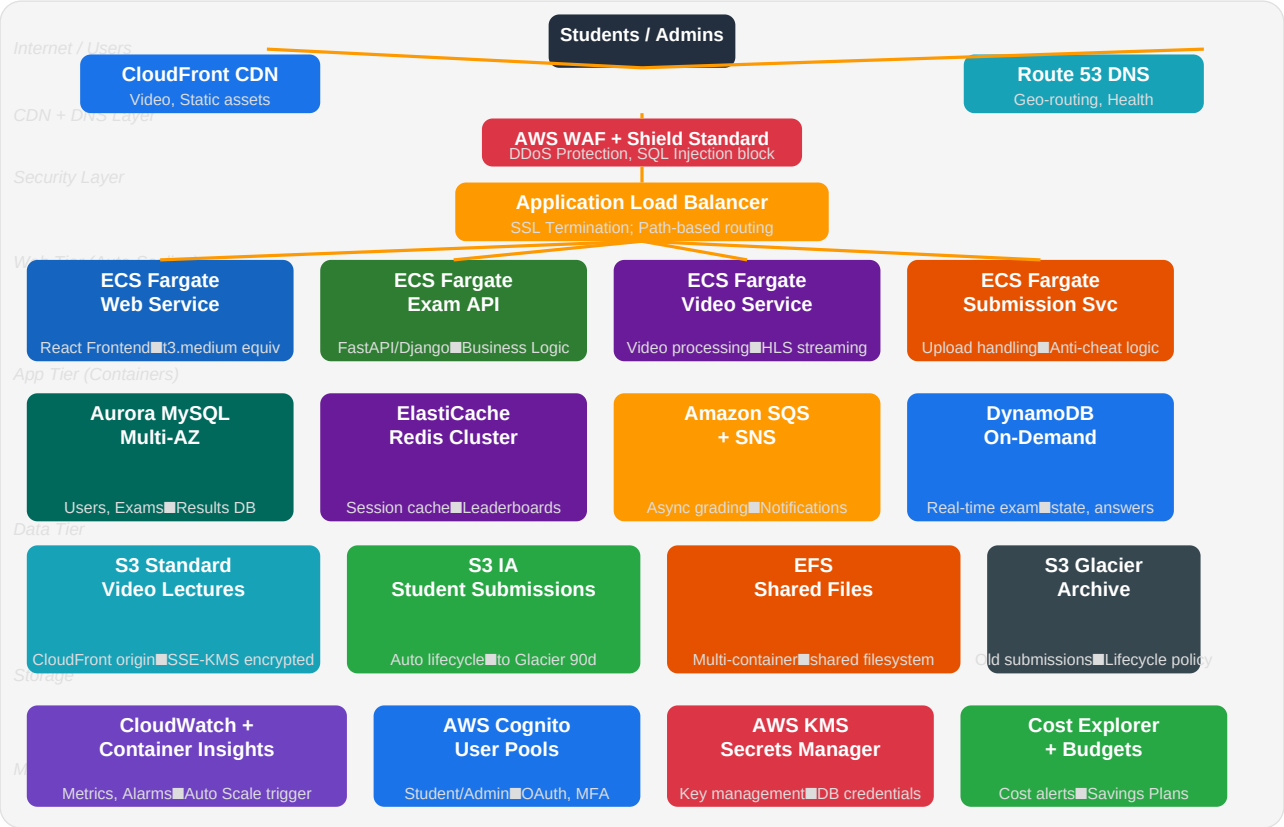
Security Best Practice: Restrict SSH inbound rule in Security Group to your IP only (MyIP). Never open port 22 to 0.0.0.0/0 in production.

Q15. XYZ Online Exam Platform — Design a Complete AWS Solution for Unpredictable Traffic

Requirements Analysis

- Unpredictable traffic spikes during exam seasons (10x–100x normal)
- Stores: video lectures, student submissions (documents, code), exam data
- High availability required — exam downtime is unacceptable
- Security — student data, exam content confidentiality
- Cost control — off-peak must be economical

Complete AWS Architecture Design



Service Mapping for XYZ Platform

Requirement	AWS Service	Justification
Unpredictable traffic	ECS Fargate + Auto Scaling	Scale containers automatically; pay per task-second
Video lectures	S3 Standard + CloudFront	Low-latency streaming via CDN; HLS for adaptive bitrate
Student submissions	S3 Standard-IA + SQS	IA reduces cost; SQS queues async grading jobs
Real-time exam state	DynamoDB On-Demand	Millisecond reads/writes; auto-scales with no capacity planning
User authentication	AWS Cognito	OAuth2/JWT; MFA support; exam role-based access
Database	Aurora MySQL Multi-AZ	High availability; automatic failover; read replicas for reports
Session management	ElastiCache Redis	Fast session lookup; prevents repeated DB auth hits
Anti-DDoS	WAF + Shield Standard	Block malicious traffic during high-profile exams

Requirement	AWS Service	Justification
Security	KMS + Secrets Manager	Encrypt data at rest; rotate DB credentials automatically
Monitoring + Alerts	CloudWatch + SNS	Alert on CPU >70%, error rate >1%, latency >500ms
Cost control	Fargate Spot + Savings Plans	Fargate Spot for batch grading (60% savings)

Scalability Plan for Exam Season

- **Pre-Exam (1 week before):** Use Scheduled Scaling — increase ECS desired count to handle projected load.
- **During Exam:** Target Tracking on CPU → ECS/EC2 ASG scales in 2–3 min; CloudFront absorbs video traffic.
- **Post-Exam (Grading):** SQS queues submissions → Lambda/Fargate Spot processes at low cost.
- **Off-Season:** ECS scales to 1-2 tasks; DynamoDB On-Demand has no idle cost; budget alerts set.